AutoScope

We read malaria slides in the cloud. OpenCV 5 does the image work, and an agent decides where to look next.

Uriel Nguefack Yefou nguefackuriel@gmail.com · github.com/nguefackuriel
Alex Bruno Kenfack Anafack kenfack_bruno@outlook.fr · github.com/Br-AI

AWARD PATHS
Both. Best Use of COOL and Agentic Vision

CORE WORKLOAD
OpenCV 5 with COOL on AWS Graviton4 (c8g)

DATA
Public NLM/LHNCBC malaria slide sets. No patient data, no hardware needed

WHAT WE HAVE DONE BEFORE
One of us was an OpenCV AI Competition 2021 finalist. The other came 34th of 368 on a malaria microscopy challenge

1 The problem

Doctors still use a microscope to find malaria. They stain a drop of blood, put it under the lens, and count parasites by eye. The WHO treats this as the reference test, and it is the only routine test that gives you a **parasite count**. That count matters. It tells the doctor how sick the patient is, whether they need a drip instead of pills, and whether the drugs are working.

Malaria is getting worse, not better. The *World malaria report 2025* counted **282 million cases and 610,000 deaths in 2024**. That is 9 million more cases and 12,000 more deaths than the year before. Africa had 265 million of those cases, 94% of the world total, and 579,000 of the deaths, 95%. More than 75% of the people who died in Africa were children under five.

Two things are making rapid tests less useful, and both of them push work back onto the microscope.

The first is a genetic change called ***pfhrp2* and *pfhrp3* deletion**. Rapid tests look for a protein called HRP2. Some parasites have stopped making it, so the test says negative even though the patient has malaria. WHO had found these parasites in 42 countries by 2024, and put out a plan to deal with them in December that year.

The second is **artemisinin resistance**. Artemisinin is the main malaria drug. It still cures people, but in some places it clears the parasites more slowly than it should. Doctors noticed because too many patients still had parasites in their blood on day 3 of treatment. The only way to see this is to count parasites on day 0, day 2 and day 3. WHO now finds or suspects it in several African countries. Rapid tests are no help, because they do not count.

So more slides need reading, and there are no more people to read them. The WHO method asks a technician to look at about 100 fields under oil before calling a slide negative, then count parasites against 200 white blood cells. That is twenty to thirty minutes of close work per slide, and people make more mistakes as they get tired. African labs get the most slides and have the fewest trained readers.

AutoScope turns a normal microscope and a phone into a scanner run by an agent. The phone films through the eyepiece and uploads the video. In the cloud, OpenCV 5 checks each frame, fixes the lighting, lines up views that overlap so we never count the same parasite twice, finds the cells and sorts them. An agent reads those numbers and decides what to do next. Refocus. Move the slide. Skip a dirty field. Keep going, or stop. You get back a parasite count per microlitre, how sure we are of it, a full log of what the agent did, and a flag when it is not confident.

Three things should come out of this. Technicians spend less time per slide, because the agent stops when it has enough instead of always counting to 100. Two labs reading the same slide get the same number, which two people often do not. And we catch more of the light infections, because the agent keeps looking past 100 fields while the count is still shaky.

> **WHAT THIS IS, AND WHAT IT IS NOT**
>
> AutoScope helps a trained technician read slides. It is not a diagnostic device. A person sees every result, can check it and can change it, and when the pictures are bad the system asks for help instead of guessing. We say this in the app, in the report and on the demo page.

2 Who uses it

The people at the microscope. Lab technicians in district hospitals and rural health centres. They already have a microscope. They have no digital gear, and most of them read slides alone with nobody to check the work. AutoScope runs on what they have, plus a phone.

Who else it helps. National malaria programmes want counts they can compare between sites, and right now they cannot get them. NGOs run big screening campaigns. Drug trials need counts done the same way every time, and this is where their data often goes wrong.

Who it is for in the end. Patients in the worst hit places, mostly children under five, who get sent home when the infection is too light to spot. And patients on treatment whose day 2 and day 3 counts never get done.

3 What OpenCV 5 does here

Almost all of AutoScope is image work. The agent has nothing to think about except the numbers OpenCV 5 gives it, and the classifier is a small model sitting at the very end of a long chain of image processing. Take OpenCV out and nothing runs.

STAGE	WHAT WE USE IN OPENCV 5
1 · Pick usable frames	`variance of Laplacian for sharpness · calcHist on a grid of tiles to check the lighting is even · calcOpticalFlowPyrLK to catch camera shake. Each frame gets a score, and we keep it or drop it`
2 · Fix lighting and stain	`morphologyEx OPEN with a big kernel, or a wide GaussianBlur, to even out the background · Gray-World for colour · cvtColor to Lab plus histogram matching per channel, so slides stained darker or lighter still look alike`
3 · Line up the views	`ORB / AKAZE · BFMatcher with a ratio test · findHomography with RANSAC · warpPerspective · cv::detail stitching. This tells us where we are on the slide and how much two views overlap, so nothing gets counted twice`

STAGE	WHAT WE USE IN OPENCV 5
4 · Find the cells	`medianBlur · adaptiveThreshold · morphologyEx open and close · distanceTransform with watershed to split cells that touch · findContours · then area, roundness, extent and HuMoments to throw out platelets, stain specks and dirt`
5 · Sort them	`a small CNN on 64×64 patches, saved as ONNX and run by the new DNN engine in OpenCV 5 (over 80% ONNX coverage, FP16 built in). It runs in the same process, so we ship no other inference framework`
6 · Get the number	`parasites per 200 white cells, turned into parasites per microlitre at 8,000 WBC/µL, with an error bar from a Poisson or binomial model. That error bar is what tells the agent when to stop`

Stage 3 is the one to look at. If you know how much two neighbouring views overlap, you can count a parasite once instead of twice. When people count by hand, that is where most of the error comes from. The fix is basic geometry. It is boring, and it is why the final number can be trusted.

Why OpenCV 5 and not 4. The new DNN engine runs stage 5 inside the same process on Arm, right next to stages 1 to 4, so we ship one thing instead of two. FP16 halves the memory a batch of patches needs. And COOL speeds up exactly the Arm `imgproc` code we lean on, so our COOL numbers will mean something.

4 Agentic Vision: look, decide, act

The agent has seven tools. Six of them read numbers straight out of OpenCV 5.

TOOL	READS FROM	WHAT IT GIVES BACK, OR DOES
`assess_field()`	Stages 1 and 2	`{focus_score, illumination_uniformity, cell_density, debris_ratio}`
`refocus(dz)`	none	**Acts**: asks for a different focus
`move_stage(dx, dy)`	Stage 3	**Acts**: asks for a new view, and stage 3 checks it really is new
`count_field()`	Stages 4 and 5	White cells, parasites, and how sure it is about each one
`estimate_parasitaemia()`	Stage 6	The count, and how wide the error bar is
`decide_stop()`	Stage 6	Whether we have looked at enough
`flag_for_human(reason)`	none	Hands it to a person, with the evidence

The log the rules ask for: an OpenCV 5 number changing what happens next

```
field 07  assess_field()   → focus_score = 42    (needs 120)      ✗
          agent            → refocus(+2)
field 07  assess_field()   → focus_score = 310                     ✓
          agent            → count_field()    → 0 parasites / 31 WBC

field 08  assess_field()   → debris_ratio = 0.71  (limit 0.35)      ✗
          agent            → move_stage(+1, 0)     [field skipped, not counted]

after 34 fields            → estimate_parasitaemia() = 4,180 /µL, error bar 18%
                             (we want 20% or less)
          agent            → decide_stop() → STOP
                             [the fixed method would have read all 100 fields]
```

The agent is doing real work. Without the stopping rule there is no product. The fixed method costs the same whether the slide is full of parasites or clean. On a heavy slide the agent stops early and saves three to five times the compute and the technician's time. On a light slide it does the opposite and keeps looking past 100 fields, which is where the fixed method misses people. We measure both, and report them in §7.

We kept the agent on a short leash. It picks where to look and when to stop. It never picks the diagnosis. Step Functions sets the limits on fields, time and money, not the model. Anything it is unsure about goes to a person.

5 How it runs on AWS

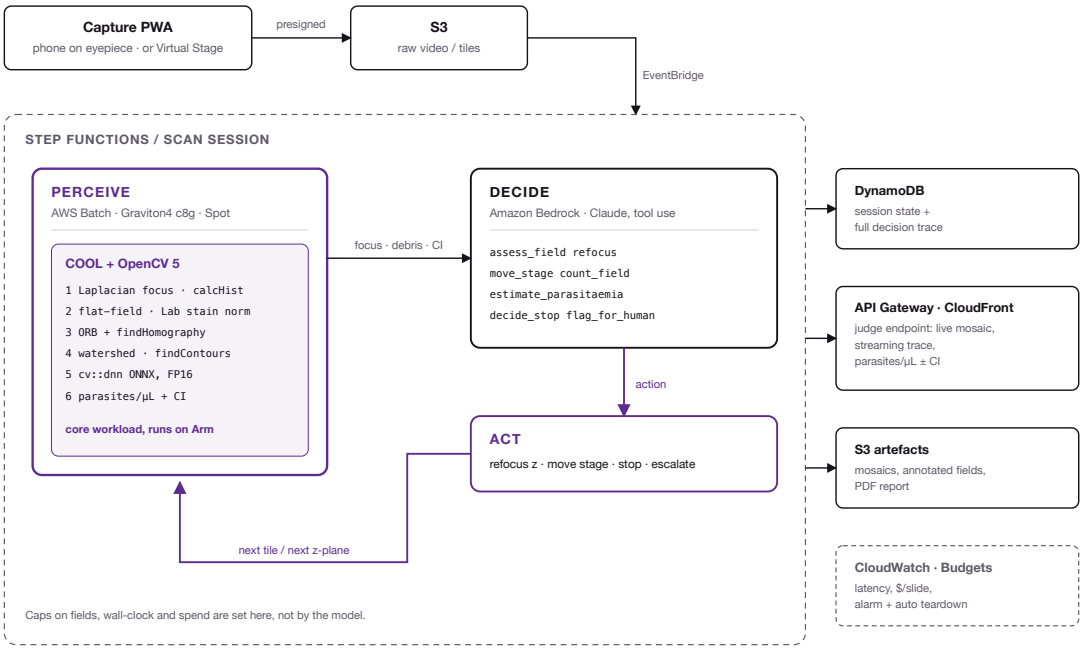


Figure 1. The violet path is where both special awards live. Stages 1 to 4 and 6 run on Graviton with COOL. The arrow that loops back is the agent part: what OpenCV measures decides what gets looked at next.

PART	SERVICE	WHAT IT DOES
Upload	S3 presigned upload	A web app on the phone uploads the video or the tiles. Nothing sits between the phone and S3
Start	EventBridge	A new file starts a reading session
Control	Step Functions	Runs the look, decide, act loop, keeps the session state, and holds the limits
The image work	**AWS Batch on Graviton4 (c8g), Spot**	A container built from the COOL AMI, running OpenCV 5. Stages 1 to 4 and 6 run here. **This is the main workload, and it runs on Arm.**
The model	OpenCV 5 DNN, same container	No second endpoint to set up, pay for or measure
The agent	Amazon Bedrock (Claude, tool use)	Picks the next move from OpenCV's numbers. We save every call and everything it saw
State and logs	DynamoDB	Session state, the numbers from every field, and the full log of decisions
Files	S3	Stitched slides, marked up fields, PDF reports
For the judges	API Gateway with Lambda; CloudFront with S3	Public page and dashboard: the slide building up, the log scrolling, the count and its error bar
Running it	CloudWatch and AWS Budgets	Speed, cost per slide, alarms, and shutting things down between test runs

Money. We are opening a new AWS account, so all we have is the Free Tier credit and the competition grant. The design says as much. Batch runs on Spot Graviton. We save the COOL trial for the week we run the benchmark. Bedrock only makes the small decisions, so the heavy work stays in OpenCV and never goes through the model. A budget alarm shuts things down on its own. Every service in that table has a job.

6 Best Use of COOL: what we will measure

Stages 1 to 4 and stage 6 run in a container built from the **COOL AMI on AWS Graviton4**. That is the loop that cleans up each tile and finds the cells, and it uses most of the compute.

The COOL benchmark covers 78 `imgproc`, `core` and I/O functions. Gains average over 1.2×, and about 1.8× on the common image processing calls. The functions we lean on are `resize`, `medianBlur`, `GaussianBlur`, `adaptiveThreshold`, `morphologyEx`, `findContours`, `calcHist` and `warpPerspective`. They are all in that set, so the speedup should show up across the whole pipeline and not just in a microbenchmark.

What we will publish, with the script and the Terraform

- **Timings for each function**, on our real tile sizes and types, COOL against plain OpenCV 5, both on `c8g`. We give the median and the spread over at least 100 runs. Not the best run.
- **Speed of the whole pipeline**: slides per hour, and fields per second.
- **Cost**: dollars per 1,000 slides on `c8g` with COOL, `c8g` without it, and `c7i` on x86, all at the same accuracy. That is the number a lab cares about.
- **Proof COOL is doing the work**: which build we used, which library version got linked, and a check written into the log of every benchmark run.

We are putting the x86 numbers in on purpose. If we only showed you Graviton, you would have to take our word for it.

7 How we will test it

Data: all public, properly licensed, and none of it needs hardware

- **NLM / LHNCBC Malaria Screener**, thick smear (150 patients) and thin smear. Names removed, and labelled by expert readers at the Mahidol-Oxford Tropical Medicine Research Unit. These pictures were taken with a phone held to a microscope eyepiece, which is exactly how AutoScope works.
- **Tanzania blood smear set (2024)**, kept aside, to check we still work in another country.

The Virtual Stage

We do not own a microscope, and we do not need one. First we stitch each public slide into one big image, using the same OpenCV code the pipeline uses. Then we put an API on top of it. `move_stage(dx, dy)` gives back a new view, and `refocus(dz)` gives back a different focus. So the agent really is moving around a slide, and the slide is real and labelled.

We think this beats a microscope on a bench. Everything the agent does can be replayed, so a judge can run it on their own machine and get the same log. A real microscope never gives you that. If we get hold of one during the build we will add a live demo on top, but nothing depends on it.

What we measure

QUESTION	HOW WE ANSWER IT
Does it find parasites?	Precision, recall and F1 per field, against the expert labels
Is it safe to screen with?	Sensitivity and specificity per patient, **split by how heavy the infection is**
Is the number right?	Our count per microlitre against the expert count: a regression, plus **Bland-Altman** limits
Does the agent help?	Fields read against the fixed 100-field method, **at the same sensitivity**
Does it work elsewhere?	All of the above, run again on the Tanzania set we kept aside
Does COOL pay off?	Speedup per function, slides per hour, dollars per 1,000 slides (§6)

The one we care about most is the Bland-Altman agreement on the count. Most work on automated microscopy reports a classification F1 and skips it. The count is what changes treatment, so that is what we want to be judged on.

What goes wrong. We will measure it and publish it, because the rules ask for it and because this is health software. Slides stained too dark or too light. Platelets and stain specks that look like parasites. Fields that will not focus. Infections under 100 parasites per microlitre. Species other than *falciparum*. Badly made slides. Each one gets a number, not a sentence.

Running it responsibly. A person stays in the loop. We claim nothing about diagnosis. There is a clear path to a human, a model card saying what it is for, what it is not for and where it breaks, only public data with names removed, nothing personal stored, and the cost and the delays written down.

8 What the judges will see

We will keep a public page on CloudFront and API Gateway up for the whole judging period. A judge picks a slide and watches it run:

- the slide building up view by view as the agent moves around it;
- the log beside it, showing which OpenCV number caused each refocus and each skip;
- the count and its error bar getting tighter until the agent stops;
- a PDF to download, and a one click COOL against plain comparison for that run.

We will put a hard slide one click away, one with few parasites and bad staining, so a judge can watch it struggle and ask for help without going looking for it. There is also a five minute video, a repo with pinned versions and Terraform, and the written report.

9 Who we are

Two engineers from Cameroon, both living in Montreal. Between us: a finalist place in the OpenCV AI Competition, a top 10% finish on a malaria microscopy challenge, computer vision shipped for medical imaging, and eight years of Python in production. We each plan on about 20 hours a week for the nine weeks.

Uriel Nguefack Yefou · the OpenCV pipeline and the testing

MSc Data Science, AIMS Cameroon (3.54/4) · Machine learning engineer, Montreal · Zindi Africa Country Ambassador

At YULCOM Technologies he led three developers building computer vision for **medical imaging**, and got model accuracy up by 25%. He also put NLP and OCR systems into production with Docker. Before that he was an AI Research Engineer at the **Montreal Heart Institute**, working on explainable AI for ECG data, and presented it at the CMDO Symposium at MILA. Earlier he built systems in Cameroon that spot, count and sort streetlights in photos.

Competitions. 68 on Zindi. The one that counts here is the **Lacuna Malaria Detection Challenge, where the job was sorting parasites and blood cells in microscope pictures from malaria patients. He came 34th of 368.** He also came 7th of 110 on Kuyesera, finding damaged buildings in aerial photos, 17th of 344 on Ghana Crop Disease Detection, 16th of 203 on the UNIDO AfricaRice check, and 11th of 32 on the Cameroon kidney ultrasound challenge.

Community. Zindi Africa Country Ambassador. Ran the 2024 IndabaX Cameroon Hackathon. Reviews for the 8th Black in AI Workshop at NeurIPS 2024, AI4Ed at AAAI 2024 and 2025, WiML at NeurIPS 2025, and the African Computer Vision Summer School in 2024 and 2025.

Tools. PyTorch, OpenCV, CNN, ResNet, ViT, SHAP. AWS with EC2, Lambda, S3 and IAM, plus Docker. LangChain, LangGraph and CrewAI. ISTQB Certified Tester, Foundation Level.

`github.com/nguefackuriel` · `linkedin.com/in/uriel-nguefack-yefou` · `zindi.world/users/urielnguefack`

Alex Bruno Kenfack Anafack · AWS, the COOL benchmark, the agent

BSc Computer Science, University of Dschang · Python Developer, Equisoft, Montreal · Founder and CEO, AI Technologies for Africa (AITECAF)

Eight years of Python in production, across backend APIs, data pipelines, computer vision and generative AI. At Equisoft he builds and looks after the Python core of a life insurance platform: REST and GraphQL, RabbitMQ with dynamic queues and failover, tracing, and the Azure DevOps pipeline several teams rely on. He also started and runs AI Technologies for Africa. AITECAF built computer vision, IoT and RAG products for **Expertise France**, including fraud detection tooling for the Audit Chamber of the Supreme Court of Cameroon, and for the **Embassy of Spain**. It also shipped Peniang, which has six parts live, two of them Android apps on Google Play.

Competitions and grants. **Finalist in the OpenCV AI Competition 2021, phase 1**, for spotting plant disease with Fast R-CNN, PyTorch and OAK-D. The team won four OAK-D cameras. He won the IndabaX Cameroon Ideathon in 2023 with a pothole detector built on PyTorch and YOLOv8, and came second in the 2021 IndabaX Cameroon competition on Zindi with 0.9973, the same score as first place, split on a tiebreak. He raised about 3,500,000 FCFA in public innovation grants from Fonds Proto and TYSYP, plus a £5,000 Afri-Plastics grant.

Tools. OpenCV, OpenVINO, PyTorch, YOLOv8, Fast R-CNN, Meta SAM, edge AI. Docker, Terraform, GitHub Actions, Azure DevOps, Amazon S3. FastAPI, Django, PostgreSQL. Contributes to `scikit-image`, `Ivy` and Mozilla's `bugbug`. Has the OpenCV University certificate in Computer Vision and Image Processing, Microsoft Azure AI Fundamentals (AI-900), and the Intel Edge AI for IoT Developers Nanodegree.

`github.com/Br-Al` · `linkedin.com/in/bruno-alex-kenfack-anafack` · `br-al.github.io/kenfackanafack`

Who does what. Uriel takes the OpenCV 5 pipeline and the testing, since the malaria and medical imaging work is his. Alex takes AWS, the Graviton and COOL benchmark, and the agent, since the production

engineering and the earlier OpenCV competition are his. We write the report, build the demo page and make the video together.

10 Nine weeks, two people

WEEKS	WHAT WE FINISH	WHAT IT SETTLES
1 to 2	OpenCV 5 pipeline, stages 1 to 4 and 6, running on our laptops on the NLM data. Virtual Stage built. First numbers on the board	Can we do the image work at all
3	Classifier trained and running under the OpenCV 5 DNN engine, with numbers per field	Do we find the parasites
4 to 5	Graviton and COOL container, AWS Batch, S3, Step Functions. **Mid-point check-in**	Does it run on AWS, and on Arm
6	Bedrock agent, seven tools, logs in DynamoDB, stopping rule, and the measurement of what the agent saves	Does the agent earn its place
7	Dashboard, public page, phone app	What the judges touch
8	COOL benchmark runs, full testing, the cases where it breaks	The COOL numbers, and the proof
9	Report, five minute video, repo cleanup, spare time	Nothing missing when we submit

The two things that could go wrong for us are Arm with COOL, and the agent loop. Both are done by week 6. That leaves three weeks to gather proof and fix problems, instead of three weeks of hoping.

11 Which paths we are entering

Both. Best Use of COOL, in §6, and Agentic Vision, in §4.

Where our numbers come from. All the malaria numbers are from the WHO *World malaria report 2025*, which covers 2024: 282 million cases and 610 000 deaths worldwide; 265 million cases (94%) and 579 000 deaths (95%) in the WHO African Region; more than 75% of the African deaths in children under five. The same report gives 42 countries with *pfhrp2* deletions as of 2024, the December 2024 response plan, and the account of artemisinin resistance and day 3 parasites. The 100 fields and the 200 white cells come from the WHO malaria microscopy standard operating procedures, not from that report. NLM/LHNCBC malaria data at lhncbc.nlm.nih.gov/LHC-downloads/dataset.html. COOL at opencv.org/cool, plus the AWS Graviton COOL benchmark. Competition results can be checked at zindi.world/users/urielnguefack/competitions/portfolio.